

Tech Plus Learnings — Docker Complete Course

Docker Complete Course — Commands Reference

Every command used in the video, in the same order we used them. Follow it top to bottom and you'll rebuild everything from the course: basic containers, port mapping, volumes, a Node.js/Express app, a Python FastAPI app, and a Docker Compose setup.

1. Install & Verify Docker

Windows only — install WSL first (Docker runs on Linux under the hood), then restart your computer:

```
wsl --install
```

Then install **Docker Desktop** for your OS (Windows / Mac Apple Silicon / Mac Intel / Linux) from the official Docker website and open it once.

Verify the installation:

```
docker -v
# or
docker --version
```

If you see a version number, Docker is installed correctly.

2. Your First Container

Run an Ubuntu container in interactive mode:

```
docker run -it ubuntu
```

- If the `ubuntu` image isn't on your machine, Docker **pulls it from Docker Hub** automatically.
- `-it` = **interactive terminal** — it drops you inside the container.

Commands we ran **inside** the container:

```
ls          # list folders inside the container (Linux file system)
mkdir test  # create a folder — it exists only in THIS container
exit        # leave and stop the container
```

Run `docker run -it ubuntu` in two terminals and you get **two separate containers** from the same image. A folder created in one does not appear in the other — that's **container isolation**.

3. List Containers

```
docker ps          # running containers only
docker ps -a       # all containers (running + stopped)

docker container ls # same as docker ps
docker container ls -a # same as docker ps -a
```

4. Remove Containers & Images

```
docker rm <container_name_or_id>    # remove a container
docker images                        # list all images
docker rmi <image_id>                # remove an image
```

Remember: `rm` removes **containers**, `rmi` removes **images**.

5. Name Your Container, Start It Again & Enter It

Give the container your own name instead of a random one:

```
docker run -it --name my-os ubuntu
```

Running the same command again gives a **name conflict error**, because `docker run` always creates a **new** container. Two ways to fix it:

Option A — delete the old container and run again:

```
docker rm my-os
docker run -it --name my-os ubuntu
```

Option B — reuse the existing container (recommended):

```
docker start my-os          # start the stopped container (runs in background)
docker exec -it my-os bash  # enter the running container
```

`docker exec` runs any command inside a **running** container. Without `-it bash`, it just runs the command and returns:

```
docker exec my-os ls        # list files inside my-os
docker exec my-os mkdir hello # create a folder inside my-os
```

Command	What it does
<code>docker run</code>	Creates a new container from an image and starts it
<code>docker start</code>	Starts an existing stopped container
<code>docker exec -it <name> bash</code>	Opens a terminal inside a running container

6. Auto-Remove Containers with `--rm`

The container is deleted automatically as soon as it stops — no name conflicts, no leftover containers:

```
docker run -it --rm --name my-os ubuntu
exit    # container stops AND gets removed
```

Flags always go **before** the image name.

7. Port Mapping (Nginx Demo)

A server running inside a container is isolated — you can't reach it from your browser until you **map a port**.

```
docker run -it -p 9000:80 --name my-server nginx
```

- `-p <your_machine_port>:<container_port>`

- `9000` = port you open in the browser
- `80` = port Nginx runs on inside the container

Open in browser: <http://localhost:9000> → "Welcome to nginx!"

Detached mode — keep it running after closing the terminal

```
docker rm my-server          # remove the old one first
docker run -d -p 9000:80 --name my-server nginx
```

- `-d` = **detached mode** — runs in the background, prints only the container ID.

Stop and start it again:

```
docker stop my-server
docker start my-server
```

8. Pull Images & Use Tags

```
docker pull python          # latest version
docker pull python:3.13-slim # a specific tag/version
```

Run a container from a specific tag:

```
docker run -it python:3.13-slim # opens the Python shell
```

Press `Ctrl + D` to exit.

Find all available tags on the image's page on **Docker Hub**.

9. Volumes — Persistent Data

Without volumes, data inside a container is **lost when the container is removed**. A volume links a folder on your machine to a folder inside the container.

```
docker run -it -v <your_local_path>:/home/data --name volume-container ubuntu
```

Example (Mac/Linux):

```
docker run -it -v /Users/<your-username>/Desktop/docker-course/data:/home/data --name volume-container ubuntu
```

Example (Windows):

```
docker run -it -v C:¥Users¥<your-username>¥Desktop¥docker-course¥data:/home/data --name volume-container ubuntu
```

Inside the container:

```
cd home
cd data
touch db-backup.json # appears in your local data folder too
mkdir database       # also appears locally
exit
```

Now start a **completely new container** pointing at the same local folder:

```
docker run -it -v /Users/<your-username>/Desktop/docker-course/data:/home/users ubuntu
cd home/users
ls # all the old files are still here
```

The container path can be different (`/home/users`) — what matters is the **local path**, which is where the data actually lives.

10. Dockerize a Node.js / Express App

10.1 Create the app

```
node -v          # make sure Node.js is installed
mkdir express-api
cd express-api
npm init         # creates package.json
npm i express    # install Express
```

Create `index.js`:

```
const express = require("express");

const app = express();
const port = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.send("Hello from Express Application");
});

app.listen(port, () => {
  console.log(`Server is running at http://localhost:${port}`);
});
```

Run it locally to test:

```
node index.js
```

Open <http://localhost:3000>.

10.2 Write the Dockerfile

Create a file named `Dockerfile` (no extension) inside `express-api/`.

Optimized version (uses layer caching — dependencies are only reinstalled when `package.json` changes):

```
# Base image (Node.js on Linux)
FROM node:24-alpine

# Create /app and move into it (mkdir app + cd app)
WORKDIR /app

# Copy dependency files first — this layer stays cached
COPY package.json package-lock.json ./

# Install dependencies
RUN npm install

# Copy app code last — this changes most often
COPY index.js ./

# Documents the port (for developers — doesn't publish it)
EXPOSE 3000

# Command that starts the app
CMD ["node", "index.js"]
```

Why this order? Docker caches each step. If you only change `index.js`, Docker reuses the cached `npm install` layer and the rebuild takes seconds. Put things that rarely change (dependencies) first and things that change

often (your code) last.

10.3 Build the image

```
docker build -t node-api .
```

- `-t node-api` = image name (tag)
- `.` = the Dockerfile is in the current folder

Rebuild without using cache:

```
docker build --no-cache -t node-api .
```

10.4 Run the container

Interactive mode:

```
docker run -it -p 4000:3000 --rm --name api-server node-api
```

Detached mode:

```
docker run -d -p 4000:3000 --rm --name api-server node-api
```

Open <http://localhost:4000> (not 3000 — 3000 is the port *inside* the container).

Stop it (it auto-deletes because of `--rm`):

```
docker stop api-server
```

11. Dockerize a Python FastAPI App

11.1 Create the app

```
mkdir python-api
cd python-api

python -m venv env

# Mac / Linux
source env/bin/activate
# Windows
env\Scripts\activate

pip install "fastapi[standard]"
```

Create `app.py`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to FastAPI Application"}

@app.get("/about")
def about():
    return {"message": "This is about page"}
```

Run it locally to test (either command works):

```
fastapi run app.py
# or
uvicorn app:app --port 8080
```

Save your dependencies:

```
pip freeze > requirements.txt
```

11.2 Write the Dockerfile

Create `Dockerfile` inside `python-api/`:

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 8080

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Important: `--host 0.0.0.0` is required. Without it, uvicorn listens only on `127.0.0.1` inside the container and the app is unreachable from your browser even with port mapping.

11.3 Build & run

```
docker build -t fastapi-image .
docker run -d --rm -p 8081:8080 --name fastapi-server fastapi-image
```

Open <http://localhost:8081> and <http://localhost:8081/about>.

12. Docker Compose — Run Multiple Services Together

Create `docker-compose.yml` in the root `docker-course/` folder (the one containing `express-api/` and `python-api/`):

```
services:
  postgres:
    image: postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password

  redis:
    image: redis
    ports:
      - "6379:6379"

  nginx:
    image: nginx
    ports:
      - "5001:80"
```

```
express-api:
  build: ./express-api
  container_name: express-api-container
  ports:
    - "4000:4000"
  environment:
    PORT: 4000

fastapi-api:
  build: ./python-api
  container_name: python-api-container
  ports:
    - "8080:8080"
```

Notes

YAML is indentation-sensitive — use spaces, and keep a space after each `-` in `ports` .
The top-level `version:` key is obsolete in current Docker Compose, so it's left out.
`build:` points to a folder containing a `Dockerfile` ; `image:` pulls a ready-made image.
`PORT: 4000` makes the Express app listen on 4000 (via `process.env.PORT`), so the mapping is `4000:4000` .

Compose commands

```
docker compose up      # build (if needed) and start all services, logs in terminal
docker compose up -d   # same, in detached mode
docker compose down    # stop AND remove all the containers
```

Open:

- <http://localhost:4000> → Express app
- <http://localhost:8080> → FastAPI app
- <http://localhost:5001> → Nginx

Troubleshooting

Error	Cause	Fix
<code>Conflict. The container name "/my-os" is already in use</code>	<code>docker run</code> tried to create a new container with an existing name	<code>docker rm <name></code> first, or use <code>docker start <name></code> to reuse it, or add <code>--rm</code>
<code>port is already allocated</code>	Another container is already using that host port	Stop/remove the other container, or map a different host port
Site not loading from a container	Port not mapped, or app listening on <code>127.0.0.1</code>	Add <code>-p host:container</code> ; for FastAPI/uvicorn use <code>--host 0.0.0.0</code>
<code>unknown shorthand flag: 'r' in -rm</code>	Single dash used for a long flag	Use <code>--rm</code> (two dashes)

Flag ignored / treated as image name	Flag placed after the image name	Put all flags before the image name
--------------------------------------	----------------------------------	--

Quick Cheat Sheet

Command	Purpose
<code>docker -v</code>	Check Docker version
<code>docker run -it <image></code>	Create + start container, open terminal
<code>docker run -d <image></code>	Create + start container in background
<code>docker run --name <name> <image></code>	Give the container a name
<code>docker run --rm <image></code>	Auto-remove container when it stops
<code>docker run -p 9000:80 <image></code>	Map host port 9000 → container port 80
<code>docker run -v <local>:<container> <image></code>	Mount a volume
<code>docker ps</code> / <code>docker ps -a</code>	List running / all containers
<code>docker container ls -a</code>	List all containers
<code>docker start <name></code>	Start a stopped container
<code>docker stop <name></code>	Stop a running container
<code>docker exec -it <name> bash</code>	Open a terminal in a running container
<code>docker exec <name> <cmd></code>	Run a single command in a container
<code>docker rm <name/id></code>	Remove a container
<code>docker images</code>	List images
<code>docker rmi <image_id></code>	Remove an image
<code>docker pull <image>:<tag></code>	Download an image
<code>docker build -t <name> .</code>	Build an image from a Dockerfile
<code>docker build --no-cache -t <name> .</code>	Build without cache
<code>docker compose up</code> / <code>up -d</code>	Start all Compose services
<code>docker compose down</code>	Stop and remove all Compose services